AISSAI HACKATHON 2026 • AI FOR PARTICLE ACCELERATORS

# Getting Started with DALIA @ IDRIS

HPC, Slurm & Apptainer Guide for Europe's First NVIDIA GB200 NVL72 System

**Host Center: IDRIS – CNRS National Supercomputing Center (Orsay, France) | GENCI • CNRS • AI2F**

Organizers: AISSAI • IN2P3 • GANIL | Scientific Tracks: Surrogate Modeling • Anomaly Detection • Control & Optimization

# Objectives: From Zero to GPU in 30 Minutes

## 1. HARDWARE & ARCHITECTURE

**• Meet DALIA (GB200 NVL72)**
18 nodes, 72 GPUs, 5th-Gen NVLink interconnect.

**• The Critical x86 vs ARM Trap**
Login node is Intel; compute nodes are ARM Grace.

**• Filesystems & Outbound Internet**
Lustre work storage, quotas, and direct web access.

## 2. SOFTWARE & CONTAINERS

**• Using Apptainer**
Avoid compiler conflicts on ARM64 Blackwell.

**• Pre-built Hackathon Images**
PyTorch 2.x, WandB, and others ready to use.

**• Redirecting Environment Caches**
Protecting your small $HOME quota.

## 3. SLURM WORKFLOW & SCALING

**• Interactive Prototyping (srun)**
Debug models live before queuing long runs.

**• Batch Training Scripts (sbatch)**
Best practices, CPU-to-GPU ratios, walltimes.

**• Monitoring & Fair Play**
Inspecting GPU saturation with nvidia-smi & WandB.

**Target: Every team should aim to have (at least) one verified GPU session active today**

AISSAI HACKATHON 2026 • DALIA @ IDRIS

# Meet DALIA: The Next-Gen AI Supercomputer




Location: IDRIS – CNRS, Campus Université Paris-Saclay, Orsay, France

AISSAI HACKATHON 2026 • DALIA @ IDRIS

# Meet DALIA: The Next-Gen AI Supercomputer

**72**

**NVIDIA GB200 GPUs**

186 GB HBM3e VRAM per GPU with NVLink-72

**18**

**Compute Nodes**

Grace-Blackwell NVL72 liquid-cooled rack

**144**

**ARM Cores / Node**

Dual ARM Neoverse-V2 (Grace) CPUs

**500 TB**

**Lustre Parallel Storage**

DDN AI400X2 Turbo on 200 GbE network

## Why DALIA for Particle Accelerators?

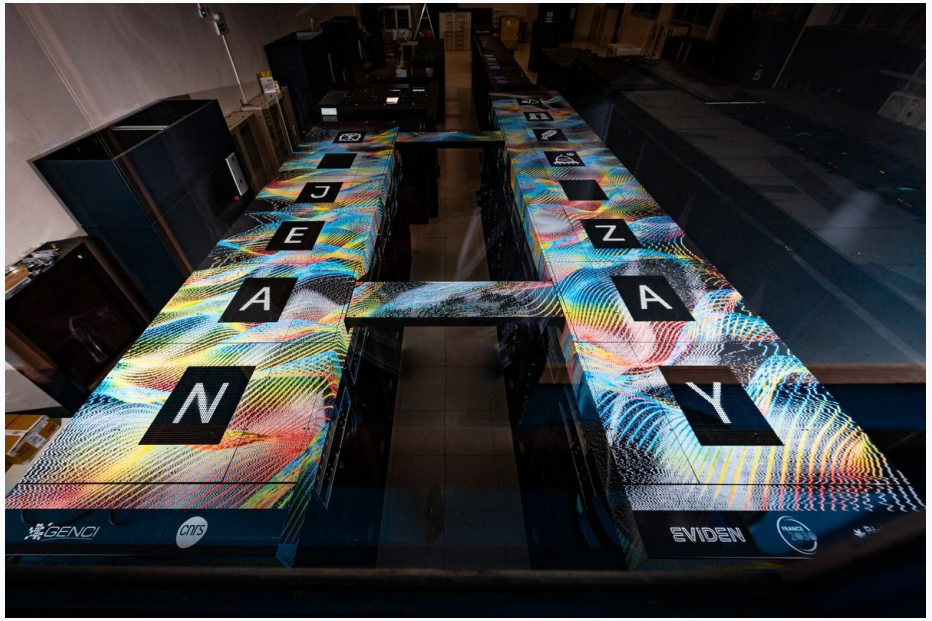
- **Europe's AI Flagship Prototype:**
  Hosted at IDRIS (CNRS supercomputing center in Orsay), acquired by GENCI to benchmark next-gen AI for open research.
- **Massive Unified Memory & NVLink:**
  NVL72 architecture allows ultra-fast inter-GPU communication, ideal for high-dimensional Surrogate Models and Graph Neural Networks.
- **Non-ZRR Machine:**
  Unlike Jean Zay which is in a restricted security zone (ZRR), DALIA features simplified onboarding for international collaborations, perfect for this hackathon.
- **Tailored for AI for Science:**
  1.7 TB of system RAM per node enables huge memory footprint for particle beam simulations, phase space modeling, and beam anomaly detection.

Location: IDRIS – CNRS, Campus Université Paris-Saclay, Orsay, France

AISSAI HACKATHON 2026 • DALIA @ IDRIS

# Meet DALIA: The Next-Gen AI Supercomputer






**Jean Zay cluster:**

- 28,880 CPUs
- 1,832 V100 GPUs
- 416 A100 GPUs
- 1,456 H100 GPus

Location: IDRIS – CNRS, Campus Université Paris-Saclay, Orsay, France

# Architecture & Pitfall #1: The CPU Mismatch

## LOGIN NODE (dalia.idris.fr)

- Architecture: Intel Xeon Gold 6530 (**x86_64**)
- Cores / RAM: 64 physical cores, 256 GB RAM
- Primary Role: SSH access, Git repository management, editing code, Slurm job submission
- Strict Rule: DO NOT run computation or heavy pre-processing here!

## COMPUTE NODES (18 Nodes)

- Architecture: ARM Neoverse-V2 / Grace CPU (**aarch64**)
- Cores / RAM: 144 physical ARM cores, 1.7 TB RAM per node
- Accelerators: 4x NVIDIA GB200 GPUs (186 GB HBM3e)
- Primary Role: Model training, inference, data loading, interactive debugging sessions

## CRITICAL PITFALL: NEVER COMPILE ON THE LOGIN NODE FOR COMPUTE NODES

- Binary Incompatibility: Login is x86_64; Compute is ARM64 (aarch64). A wheel or binary compiled on the login node will instantly crash on compute nodes with 'Illegal instruction (core dumped)'
- **Solution: Always use ARM64 Apptainer containers (or compile/install inside an interactive compute node session)**

# Filesystems, Storage & Direct Internet Access

## STORAGE DIRECTORIES & QUOTAS

**• $HOME Directory (~)**

Strict quota (3 GB, 12,000 inodes). Only personal configs (.bashrc), SSH keys, and small scripts. NO datasets, conda envs, or model checkpoints here.

**• Lustre Work Space**

500 TB parallel storage on 200 GbE. Fast I/O for accelerator datasets, heavy PyTorch weights, and container images. **Use this for all hackathon work**

**• Redirecting Tool Caches**

Always point Hugging Face, PyTorch, and Apptainer cache directories to Lustre, never $HOME.

**Your Work space: `/lustre/work/pdl18409/<login>`**
**Shared space: `/lustre/work/pdl18409/shared`**

## COMPUTE NODES HAVE INTERNET ACCESS

**• Direct Outbound Web Connection**

DALIA's compute nodes can reach the internet directly during jobs

**• Live Weights & Biases / MLflow**

Stream your training loss, validation curves, and GPU metrics directly online

**• Hugging Face & GitHub Inside Jobs**

Download pretrained transformers, tokenizers, or git submodules on the fly without offline workarounds.

**• PIP / Package Downloads**

Install missing lightweight Python libraries dynamically during interactive sessions if needed

**Try to keep your disk usage <90% in $HOME; Check with: df -h $HOME**

# SSH Connection & Authentication Setup

# SSH Connection & Authentication Setup

YOU



CC-IN2P3
(Lyon)



DALIA
(Orsay)

# SSH Connection & Authentication Setup




**...CREATE(D) AN ACCOUNT ON...**

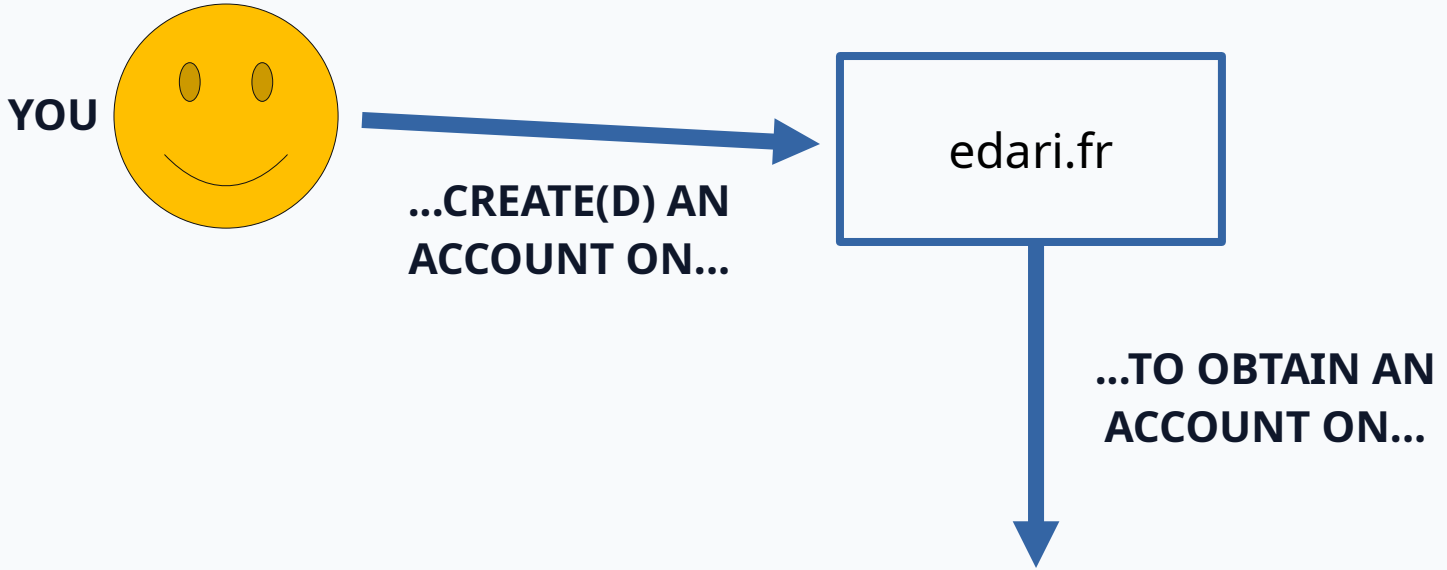


**CC-IN2P3 (Lyon)**



**DALIA (Orsay)**

# SSH Connection & Authentication Setup






**CC-IN2P3 (Lyon)**



**DALIA (Orsay)**

# SSH Connection & Authentication Setup

YOU



...WILL USE THIS MACHINE...



CC-IN2P3
(Lyon)

...TO CONNECT TO THIS MACHINE



DALIA
(Orsay)

# SSH Connection & Authentication Setup

## CONNECTION CHECKLIST

**• 1. Ed25519 Public Key:**

Use the Ed25519 key provided during hackathon onboarding.

**• 2. Whitelisted IP Address:**

Connections are only accepted from whitelisted IP addresses (either your own, your institute one, or the CC-IN2P3 one).

**• 3. Jump Host (ProxyJump):**

If relevant, route your SSH connection through an authorized gateway (your own laboratory VPN, the CC machines, etc)

ON YOUR LAPTOP OR CC-IN2P3

```
# 1. Configure your host by adding in ~/.ssh/config
Host dalia
    Hostname dalia.idris.fr
    User udlxxxxxx
    ForwardX11 yes
    ForwardAgent yes
    IdentityFile ~/.ssh/id_ed25519_dalia

# If using the CC-IN2P3 jump host, add the option:
#   ProxyJump my_cc_login@cca.in2p3.fr
# or if using another jump host:
#   ProxyJump my_jump_login@jump.machine.address


# 2. In terminal, connect directly with (passphrase will be prompted):
$ ssh dalia

[udlxxxxxx@dalia01 ~]$ # You are successfully logged in!
```

**Common issue: “Permission Denied” → Verify permissions : chmod 600 ~/.ssh/id_ed25519_dalia**

# Software Stack: Why Apptainer is King

## WHY CONTAINERS ON DALIA?

**• Architecture Isolation:**
Cleanly bundles Ubuntu ARM64 (`aarch64`), compatible Python binaries & required shared libraries; no reliance on host OS packages.

**• NVIDIA GPU Pass-through (--nv):**
`--nv` flag automatically detects host drivers and passes CUDA runtimes seamlessly into container.

**• 100% Reproducibility:**
Everyone runs the exact same environment, avoiding 'works on my machine' issues.

**• Lustre Directory Binding (-B):**
Mount your code and data folders seamlessly with `--bind $PROJECT_DIR:/workspace`.

CONFIGURATION ON DALIA

```
# 1. Add to your ~/.bashrc (MANDATORY):
export PROJECT_DIR=/lustre/work/pdl18409/$USER
export APPTAINER_CACHEDIR=$PROJECT_DIR/cache_apptainer
export APPTAINER_TMPDIR=$PROJECT_DIR/tmp_apptainer
mkdir -p $APPTAINER_CACHEDIR $APPTAINER_TMPDIR

# 2. Load Slurm automatically on login (might be there already):
module load slurm/slurm/24.11

# 3. Quick test running container inside interactive node (see later):
srun apptainer exec --nv \
  --bind $PROJECT_DIR:/workspace \
  /lustre/work/pdl18409/shared/containers/surrogate_models.sif \
  python3 -c 'import torch; print(torch.__version__)'

# Expected output: 2.10.0a0+a36e1d39eb.nv26.01.42222806

# Note that you can also `--bind` a shared project folder
```

**Warning: By default, Apptainer writes cache to $HOME/.apptainer.
If you don't redirect it to Lustre, your quota will block!**

# Ready-to-Use AI Containers & Packages

## OFFICIAL HACKATHON CONTAINERS

• **Shared Location on Lustre:**
/lustre/work/shared/containers/*.sif

• **Pre-installed Frameworks:**
PyTorch 2.x (Grace-Blackwell optimized)

• **Particle Accelerator ML Tools:**
Check the container's corresponding *.def files

• **Tracking & Utilities:**
Weights & Biases (wandb), hopsworks

## NEED ADDITIONAL PYTHON PACKAGES?

• **Option A: User Pip Mount**
Install extra wheels while inside a container, into a persistent folder on Lustre (using PYTHONUSERBASE environment variable)

**BUT may make collaboration more complicated**

• **Option B: Create another container**
Make a copy of the .def file, edit it accordingly, and build a new custom container **on an allocated node**:

**apptainer build new_container.sif new_def_file.def**

**All datasets are already on Lustre under**

`/lustre/work/pdl18409/shared/data`

**No need to re-download!**

# Slurm: Cluster Workload Manager

**REQUIRED STEP ON DALIA LOGIN NODE: `module load slurm/slurm/24.11`**

Slurm commands (sbatch, srun, squeue) are not in PATH by default. Check if this module is loaded.

## sinfo

**Inspect Node & Partition Health**

```
Usage: sinfo -o '%20P %5D %14F %8T'
```

Details: Lists node states: idle, alloc (allocated), or down.

## squeue

**Monitor Queued & Running Jobs**

```
Usage: squeue -u $USER
```

Job States: PD = Pending (waiting in queue), R = Running, CG = Completing.

## srun / salloc

**Interactive Terminal Allocation**

```
Usage: srun –reservation=pdl18409_oct2026
--nodes=1 –ntasks-per-node=1
--gpus-per-node=1 --cpus-per-task=1
--time=00:10:00 --pty bash
```

Details: Grabs an active terminal directly on a GPU node for debugging (customize arguments as needed)

## scancel

**Terminate & Release Jobs**

```
Usage: scancel <job_id>  (or scancel -u $USER)
```

Details: Immediately halts execution and releases allocated GPUs.

Tip: Set alias sq='squeue -u $USER' in your ~/.bashrc for quick status monitoring.

# Interactive Sessions for Rapid Prototyping

## WHY INTERACTIVE FIRST?

**• Instant Feedback:**

Don't submit a 3-hour batch job only to discover a missing dependency, wrong file path, or syntax error.

**• Direct Terminal Shell:**

`srun --pty bash` puts you right on a compute node with physical access to a GPU.

**• Apptainer Shell:**

Once on the node, open an interactive bash shell inside the container using `apptainer shell --nv`.

**• Exit When Finished:**

Always type `exit` when done to release the GPU for other teams (and stop consuming hours from **total 2000h total allocation**)

**STEP-BY-STEP INTERACTIVE WORKFLOW**

```
# 1. Allocate 1 GPU for 1 hour on an ARM compute node:
srun --reservation=pdl18409_oct2026 \
     --nodes=1 \
     --ntasks-per-node=1 \
     --gpus-per-node=1 \
     --cpus-per-task=36 \
     --time=01:00:00 \
     --pty bash

# 2. Now on compute node! Open Apptainer shell:
apptainer shell --nv \
  --bind $PROJECT_DIR:/workspace \
  /lustre/work/pdl18409/shared/containers/surrogate_models.sif

# 3. Inside container: Verify GPU & PyTorch:
Apptainer> python3 -c "import torch;
print('Device:', torch.cuda.get_device_name(0));
print('Memory:', torch.cuda.get_device_properties(0).total_memory / 1e9,
'GB')"
# Expected output: Device: NVIDIA GB200 | Memory: ~199.8 GB
```

**It’s always worth spending 10 minutes in an interactive session to validate your code before submitting any batch job**

# Submitting Batch Training Runs (sbatch)

## ANATOMY OF A DALIA JOB

- **`--reservation=pdl18409_oct2026`**
  Mandatory to get privileged access
- **`--nodes=1 & --gpus-per-node=4`**
  Request full-node NVLink connectivity for massive distributed training.
- **`--cpus-per-task=36`**
  36 ARM physical cores per GPU task (4 tasks x 36 cores = 144 cores per node).
  **Rule: 36 cores per GPU booked**
- **`--time=02:00:00`**
  Always specify a realistic walltime limit. Shorter requested times get scheduled faster.
- **`--output=logs/%x_%j.out`**
  Store job output (%x = job name, %j = job ID) for clean tracking.

train_model.sh (SUBMISSION SCRIPT)

```
#!/usr/bin/env bash
#SBATCH --job-name=aissai_hackathon
#SBATCH --reservation=pdl18409_oct2026
#SBATCH --output=slurm_logs/%x_%j.out
#SBATCH --error=slurm_logs/%x_%j.err
#SBATCH --nodes=1
#SBATCH --gpus-per-node=4
#SBATCH --ntasks-per-node=4
#SBATCH --cpus-per-task=36
#SBATCH --time=02:00:00

set -x

cd $SLURM_SUBMIT_DIR # define this env var beforehand in your .bashrc
mkdir -p slurm_logs

srun apptainer exec --nv \
  --bind $PROJECT_DIR:/workspace \
  /lustre/work/shared/containers/aissai_pytorch_2026.sif \
  python some_script.py --some-argument=something

# Submit with: sbatch train_model.sh
```

**Important: Always create the 'slurm_logs' folder beforehand, or Slurm will fail to write logs!**

# Scaling Up: Multi-GPU & Distributed Training

## MULTI-GPU WITHIN A NODE (4x GB200)

- **Fifth-Gen NVLink Interconnect:**
The 4 GB200 GPUs in each node communicate with ultra-high bandwidth. Inter-GPU communication is near-instantaneous.
- **PyTorch Distributed Data Parallel (DDP):**
Use torchrun or PyTorch Lightning / Hugging Face Accelerate:
- **torchrun --nproc_per_node=4 train_ddp.py**
Seamless parallel training with almost linear speedup.
- **Near-Zero Communication Overhead:**
Ideal for batch-intensive surrogate models and transformer backbones on large particle accelerator datasets.

## MULTI-NODE & THE NCCL STACK FIX

- **Multi-Node InfiniBand Network:**
18 nodes linked via non-blocking InfiniBand fat-tree network for multi-node jobs (e.g. 2 nodes = 8 GPUs).
- ⚠️ **CRITICAL FIX: Linux Stack Size Limit:**
By default, Linux sets stack size to 2 MB, which causes segmentation faults during large NCCL multi-node handshakes.
- **The Solution (IDRIS DALIA FAQ):**
Add this command at the top of your submission script:
- **ulimit -s 8192**
Increases stack size to 8 MB and prevents all NCCL communication crashes!
- **When to scale to multiple nodes?**
Validate on 1-4 GPUs first. Only scale to multiple nodes for final benchmark submissions on Thursday/Friday.

**4 GB200 GPUs provide ~744 GB of aggregated high-speed VRAM:
single-node will suffice for 90% of hackathon tasks!**

# Scaling Up: Multi-GPU & Distributed Training

## MULTI-GPU WITHIN A NODE (4x GB200)

- **Fifth-Gen NVLink Interconnect:**
The 4 GB200 GPUs in each node communicate with ultra-high bandwidth. Inter-GPU communication is near-instantaneous.
- **PyTorch Distributed Data Parallel (DDP):**
Use torchrun or PyTorch Lightning / Hugging Face Accelerate:
- **torchrun --nproc_per_node=4 train_ddp.py**
Seamless parallel training with almost linear speedup.
- **Near-Zero Communication Overhead:**
Ideal for batch-intensive surrogate models and transformer backbones on large particle accelerator datasets.

## MULTI-NODE & THE NCCL STACK FIX

- **Multi-Node InfiniBand Network:**
18 nodes linked via non-blocking InfiniBand fat-tree network for multi-node jobs (e.g. 2 nodes = 8 GPUs).
- ⚠ **CRITICAL FIX: Linux Stack Size Limit:**
By default, Linux sets stack size to 2 MB, which causes segmentation faults during large NCCL multi-node handshakes.
- **The Solution (IDRIS DALIA FAQ):**
Add this command at the top of your submission script:
- **ulimit -s 8192**
Increases stack size to 8 MB and prevents all NCCL communication crashes.
- **When to scale to multiple nodes:**
Validate on 1-4 GPUs first. Only scale to multiple nodes for final benchmark submissions on Thursday/Friday.

PLEASE IGNORE THAT PART

**4 GB200 GPUs provide ~744 GB of aggregated high-speed VRAM:
single-node will suffice for 90% of hackathon tasks!**

# Job Monitoring & Live Inspection

## 1. QUEUE & LOGS

**• Check Queue Status:**

```
squeue -u $USER (or --me)
```

**• Active Job States:**

PD: Pending (waiting in queue)
R: Running on node
CG: Completing / writing I/O

**• Stream Live Logs:**

```
tail -f slurm_logs/my_job_*.out
```

**• Check Job Efficiency:**

```
seff <job_id>
(CPU & memory efficiency after completion)
```

## 2. LIVE GPU INSPECTION

**• SSH into Running Node:**

```
ssh <assigned_node_name>
(e.g., ssh dalianvl04)
```

**• Run nvidia's "top" live:**

```
nvidia-smi
```

**• What to look for:**

• GPU-Util: Aim for >80%.
• If GPU-Util < 20%: Your PyTorch DataLoader may be CPU-bottlenecked
• Increase num_workers in DataLoader to saturate GPU.

**• Exit when inspected:**

Always exit the node session.

## 3. ONLINE DASHBOARDS

**• Weights & Biases (WandB):**

Because compute nodes have internet access, log your runs live:

**• wandb.init(project='...')**

Share interactive dashboards with your hackathon teammates.

**• Real-time Collaboration:**

Compare models across different hyperparameter sweeps simultaneously.

**Tip: Check nvidia-smi 5 minutes after launch for safety**
**E.g. GPU memory is full but compute is 0% → stuck in a Python data loop**

# Hackathon Survival Guide: Best Practices

## 1. FAIL FAST, SCALE LATER

• **Test on 5% Data First:** Run 2 epochs on a small subset to ensure loss decreases before submitting full training.

• **Short Walltimes:** Requesting 15-30 minutes gets scheduled almost instantly compared to 4 hours.

• **Iterative Development:** Confirm convergence speed early before spending compute on extensive hyperparameter grids.

## 2. FREQUENT CHECKPOINTING

• **Save Every N Epochs:** Save model weights (`checkpoint_epX.pt`) regularly inside `$PROJECT_DIR`.

• **Slurm Walltime Limits:** When your requested walltime expires, Slurm forcefully terminates the job. Don't lose hours of training!

• **Resume Capability:** Always write a `--resume` flag in your training script to pick up from the latest saved checkpoint.

## 3. GIT & TEAM WORKFLOW

• **Version Control Everything:** Use GitHub or IN2P3 GitLab. Commit early and branch often.

• **Avoid Concurrent File Editing:** Don't have 3 team members editing the same `.py` file inside the shared Lustre directory at once!

• **Decouple Code from Data:** Keep code in git repos and load datasets via clean config paths.

## 4. FAIR PLAY ON GPU RESOURCES

• **Release Idle Interactive Shells:** Always `exit` interactive sessions before going to lunch, coffee breaks, or presentations!

• **No Idle Hoarding:** 12 GPUs are shared among all teams. A GPU left idle blocks other people from making progress.

• **Check with squeue:** Verify that your jobs actually stopped after you finish your work sessions.

**CRUCIAL POINT: Point all relevant cache directories (HF_HOME, TRITON_HOME, TMPDIR, ... ) to Lustre Work space !**

# Troubleshooting Common Issues & Support

## COMMON ERRORS & QUICK FIXES

❌ **command not found: sbatch / srun**
Load the Slurm module:
-> module load slurm/slurm/24.11

❌ **Illegal instruction (core dumped)**
Architecture mismatch: you are executing x86 compiled binaries on an ARM node.
-> Use the ARM64 Apptainer container!

❌ **Disk quota exceeded**
Your $HOME is full. Clear ~/.cache and verify:
-> export APPTAINER_CACHEDIR=/lustre/work/...

❌ **CUDA Out of Memory (OOM)**
Batch size is too large for model context.
-> Reduce batch size, use torch.amp (bfloat16), or enable gradient checkpointing.

❌ **NCCL Segfault across nodes**
Linux stack size is too small for multi-node communication.
-> Add 'ulimit -s 8192' to your submission script.

## WHERE TO GET HELP?

- **On-Site Mentor in Room:**
Come see me!
- **Hackathon Agora:**
Post error logs there for (hopefully) quick answers.
- **Official IDRIS documentation:**
http://www.idris.fr/en/docs/dalia/dalia-presentation
- **Official IDRIS Helpdesk:**
assist@idris.fr
⚠️ IMPORTANT: You must put [Dalia] in the email subject line!